

UNIVERSIDAD AUTÓNOMA TOMÁS FRÍAS

Facultad de Ciencias Puras
Carrera de Ingeniería Informática

EXAMEN — TERCER PARCIAL · PRÁCTICO

Construcción de una API REST Segura

Proyecto SecureNotes · JWT Access/Refresh · Control de acceso · Sesiones — NestJS 11 · PostgreSQL · TypeORM

DATOS DE LA EVALUACIÓN

Asignatura: Seguridad de Software

Sigla: INF-781

Docente: M. Sc. Huáscar Fedor Gonzales Guzmán

Unidad temática: Autenticación y gestión de sesiones (GL7–GL8)

Modalidad: Individual · Desarrollo práctico con entrega

Puntaje total: 100 puntos

Gestión I — 2026

1. Escenario y objetivo

Construirás desde cero una API REST llamada SecureNotes: un servicio de notas personales en el que cada usuario administra únicamente sus propias notas. El objetivo del examen es que la API aplique correctamente los mecanismos de seguridad estudiados en las unidades de autenticación y gestión de sesiones (GL7 y GL8): hashing de contraseñas, autenticación con JWT de acceso y de renovación, protección de rutas, control de acceso a nivel de objeto y — para la nota completa— persistencia, rotación y revocación de sesiones con cookies httpOnly.

Competencia evaluada

Diseñar e implementar un servicio web autenticado basado en tokens, aplicando buenas prácticas en el manejo de credenciales y sesiones, y evaluando los vectores de ataque asociados al almacenamiento, la autorización y la renovación de tokens.

2. Requisitos técnicos

- NestJS 11 (CommonJS) sobre Node.js 20 LTS o superior.
- PostgreSQL 14+ con TypeORM. La opción synchronize solo puede usarse en desarrollo.
- Paquetes mínimos: @nestjs/typeorm, typeorm, pg, @nestjs/jwt, @nestjs/passport, passport, passport-jwt, @nestjs/config, argon2, class-validator, class-transformer (y cookie-parser para el bloque avanzado).
- Crea un proyecto nuevo llamado inf781-examen-securenotes. Puedes apoyarte en lo construido en la GL7 y la GL8, pero el código debe ser tuyo y estar correctamente integrado.

3. Modelo de datos mínimo

Entidad	Campos requeridos
User	id (uuid, PK) · email (único) · password (hash argon2) · createdAt
Note	id (uuid, PK) · title · content · ownerId (FK → User) · createdAt · updatedAt
RefreshToken (bloque avanzado)	id/sessionId (uuid) · userId · hashedToken · userAgent (nullable) · revoked (bool) · expiresAt · createdAt

4. Endpoints requeridos

Autenticación

Método	Ruta	Descripción	Protegido
POST	/auth/register	Registra un usuario (hashea con argon2) y emite tokens.	No
POST	/auth/login	Verifica credenciales y emite access + refresh.	No
POST	/auth/refresh	Renueva el access token a partir del refresh.	Refresh

Método	Ruta	Descripción	Protegido
GET	/auth/me	Devuelve los datos del usuario autenticado.	Access
POST	/auth/logout	Revoca la sesión actual (bloque avanzado).	Refresh
GET	/auth/sessions	Lista las sesiones activas (bloque avanzado).	Access

Notas (todas protegidas con el access token)

Método	Ruta	Descripción	Protegido
POST	/notes	Crea una nota del usuario autenticado.	Access
GET	/notes	Lista únicamente las notas del usuario.	Access
GET	/notes/:id	Obtiene una nota propia (404 si es de otro).	Access
PATCH	/notes/:id	Actualiza una nota propia.	Access
DELETE	/notes/:id	Elimina una nota propia.	Access

5. Requisitos de seguridad obligatorios (núcleo)

Estos requisitos constituyen el núcleo de la evaluación y deben cumplirse íntegramente:

1. Las contraseñas se almacenan hasheadas con argon2. Nunca se guarda ni se devuelve la contraseña en texto plano.
2. Toda entrada se valida con DTOs (class-validator) y un ValidationPipe global con whitelist: true y forbidNonWhitelisted: true.
3. Se emiten un access token de vida corta (p. ej. 15 min) y un refresh token de vida larga (p. ej. 7 días), firmados con secretos distintos, largos y aleatorios cargados desde variables de entorno.
4. Las rutas protegidas usan estrategia + guard. Un access token ausente, inválido o expirado devuelve 401 Unauthorized.
5. Control de acceso a nivel de objeto: un usuario no puede leer, modificar ni eliminar notas de otro. Las operaciones sobre notas ajenas devuelven 404 (no se debe filtrar la existencia del recurso).
6. El archivo .env está excluido del repositorio (.gitignore) y se incluye un .env.example sin secretos reales.
7. Ninguna respuesta de la API expone el campo password (ni su hash).
8. El endpoint /auth/refresh entrega un nuevo access token válido sin pedir credenciales otra vez.

6. Gestión segura de sesiones (bloque avanzado)

Estos requisitos elevan la API del modelo stateless de la GL7 al modelo stateful de la GL8 y son necesarios para obtener el puntaje completo:

1. Se persiste un hash del refresh token, con una fila por sesión/dispositivo en la tabla de sesiones.
2. Rotación: cada uso del refresh emite un par nuevo e invalida el refresh anterior (se sobrescribe el hash guardado).
3. Detección de reúso: presentar un refresh ya rotado se interpreta como robo y revoca todas las sesiones del usuario.
4. Logout que revoca la sesión del lado del servidor y endpoint que lista las sesiones activas del usuario.
5. El refresh token se entrega en una cookie httpOnly + Secure (en producción) + SameSite=strict, con el path acotado a las rutas de autenticación.

7. Escenarios de verificación

Debes demostrar con un cliente HTTP (Postman, Bruno o REST Client) que tu API supera estos escenarios. Incluye capturas o la colección de pruebas:

- a) Registro e inicio de sesión devuelven los tokens esperados.
- b) Una petición a /notes sin access token responde 401.
- c) Tras crear varias notas, GET /notes lista solo las del usuario autenticado.

- d) El usuario B intenta GET, PATCH o DELETE sobre una nota del usuario A y recibe 404.
- e) Con el access token expirado se obtiene 401; tras /auth/refresh y reintentar, la respuesta es 200.
- f) (avanzado) Reutilizar un refresh ya rotado revoca todas las sesiones del usuario.
- g) (avanzado) Tras el logout, un /auth/refresh posterior de esa sesión falla.

8. Entregables

- Repositorio Git inf781-examen-securenotes funcional, sin node_modules, con .gitignore y .env.example.
- README con los pasos de ejecución: instalación de dependencias, configuración del .env, base de datos y arranque (npm run start:dev).
- Colección de pruebas (Postman/Bruno/.http) o capturas de pantalla que evidencien los escenarios de verificación de la sección 7.

Condiciones del examen

Trabajo individual. Se permite consultar la documentación oficial de NestJS, TypeORM y Passport y tus propias guías GL7–GL8.

9. Rúbrica de evaluación

Criterio	Qué se evalúa	Puntos
Configuración y modelo	Proyecto NestJS + PostgreSQL operativo; entidades User y Note correctas.	10
Registro y login	Hashing con argon2, verificación de credenciales y validación con DTOs.	15
Tokens y rutas protegidas	Emisión de access/refresh con secretos distintos; estrategias y guards; 401 correcto.	15
CRUD de notas	Crear, listar, obtener, actualizar y eliminar notas funcionando.	15
Control de acceso (anti-IDOR)	El usuario solo accede a sus notas; operaciones sobre notas ajenas devuelven 404.	15
Sesiones: persistencia, rotación y reúso	Hash del refresh por sesión, rotación y revocación global ante reúso detectado.	15
Cookies, logout y sesiones	Cookie httpOnly/Secure/SameSite, logout que revoca y listado de sesiones activas.	10
Buenas prácticas y entrega	.env excluido, password no expuesta, README y pruebas demostradas.	5
	TOTAL	100

Niveles de logro

Núcleo (criterios 1–5 y 8 ≈ 75 pts): API segura nivel GL7 — autenticación, rutas protegidas y control de acceso por propietario.

Completo (criterios 6–7 ≈ 25 pts): gestión stateful de sesiones nivel GL8 — persistencia, rotación, reúso, revocación y cookies httpOnly.

Bibliografía de referencia

RFC 7519 (JWT) · NestJS — Authentication & Authorization · OWASP — JWT, Session Management y Broken Object Level Authorization (BOLA/IDOR) · Auth0 — Refresh Token Rotation and Reuse Detection · MDN — Set-Cookie: HttpOnly, Secure y SameSite.